Estrutura de Dados

Aula 02

Prof. Luiz Antonio Schalata Pacheco, Dr. Eng.

Instituto Federal de Santa Catarina
Câmpus Garopaba
Curso Superior de Tecnologia em Sistemas para Internet

schalata@ifsc.edu.br

04/03/2026

Notação Big-O

Complexidade de Algoritmos

- Como comparar dois algoritmos?
- Comparação objetiva entre algoritmos: independente de diferenças de hardware, sistema operacional e linguagem de programação
- Quanto a complexidade do algoritmo aumenta conforme as entradas crescem?

Exemplificando

- Programe, em Python, uma função para calcular o somatório de 1 até n. O valor n deve ser passado como parâmetro. Por exemplo, se a entrada for 10, o resultado será $1 + 2 + 3 + \ldots + 10 = 55$.

Versão 1

```
# Usando for
def soma1(n):
  soma = 0
  for i in range(n + 1):
      soma += i

  return soma
```

Medindo o tempo de execução

```python
import timeit

# Usando for
def soma1(n):
  soma = 0
  for i in range(n + 1):
      soma += i

  return soma

# Verificando os tempos de execucao com timeit
tempo_inicial = timeit.default_timer()
soma1(10)
tempo_final = timeit.default_timer()
print(f'Tempo soma1: {tempo_final - tempo_inicial}')
```

Aumentando o valor do parâmetro de entrada

- Execute novamente o algoritmo com $n = 1.000.000$.
- Depois execute com $n = 100.000.000$.
- Quais conclusões podem ser tiradas?
- Esse algoritmo é eficiente?
- É possível acelerar a execução?

Versão 2: Usando a matemática

```
1 # Usando equacao matematica
2 def soma2(n):
3   return (n * (n + 1)) / 2
```

Medindo os tempos

```
import timeit

# Usando equacao matematica
def soma2(n):
  return (n * (n + 1)) / 2

  return soma

# Verificando os tempos de execucao com timeit
tempo_inicial = timeit.default_timer()
soma1(10)
tempo_final = timeit.default_timer()
print(f'Tempo soma2: {tempo_final - tempo_inicial}')
```

Análises

- Observe que podemos comparar porque estamos executando no mesmo hardware e SO.
- A função `soma1` executa $n + 1$ passos.
- A função `soma2` realiza multiplicação, adição e divisão — três operações, independentes de n.
- Diz-se que a função `soma1` tem complexidade $O(n)$.
- A função `soma2` tem complexidade $O(1)$ (constante).

Resumindo

- A análise Big-O independe de hardware e SO.
- Os tempos medidos seriam diferentes em hardwares distintos; a notação Big-O torna a comparação objetiva.
- `soma1` tem desempenho ruim pois depende de n: quanto maior n, maior o tempo de execução.
- A complexidade de `soma1` cresce linearmente com as entradas.
- A função `soma1` pode não ser escalável quando n é muito grande.
- Essa é a ideia básica da notação Big-O para comparar algoritmos.
- Big-O descreve o pior caso ou o limite superior do crescimento de tempo/espaço.

Exemplo usando listas

- Criar uma lista com 1000 elementos (de 0 a 999).
- Medir o tempo de criação da lista.

Versão 1: De forma manual

```
1 def lista1():
2   lista = []
3   for i in range(1000):
4       lista += [i]
5   return lista
6
7 print(lista1())
```

Versão 2: Usando funções pré-definidas

```
1 def lista2():
2   return range(1000)
3
4 print(lista2())
5
6 l = lista2()
7 for i in l:
8   print(i)
```

Análises

- A função `lista1()` é mais lenta, pois a inserção dos valores é feita manualmente.
- Na função `lista2()`, a função `range` realiza esse processo automaticamente.
- A complexidade de `lista1()` é $O(n)$, pois há n inserções.
- A complexidade de `lista2()` depende da implementação de `range`; em geral, é otimizada.
- Pelo tempo de execução, percebe-se que `lista2()` é mais eficiente.
- Funções pré-definidas normalmente já foram otimizadas.

Funções Big-O

- Constante: $O(1)$
- Logarítmica: $O(\log n)$
- Linear: $O(n)$
- Log-linear: $O(n \log n)$
- Quadrática: $O(n^2)$
- Cúbica: $O(n^3)$
- Exponencial: $O(2^n)$

Gerando gráfico das funções Big-O

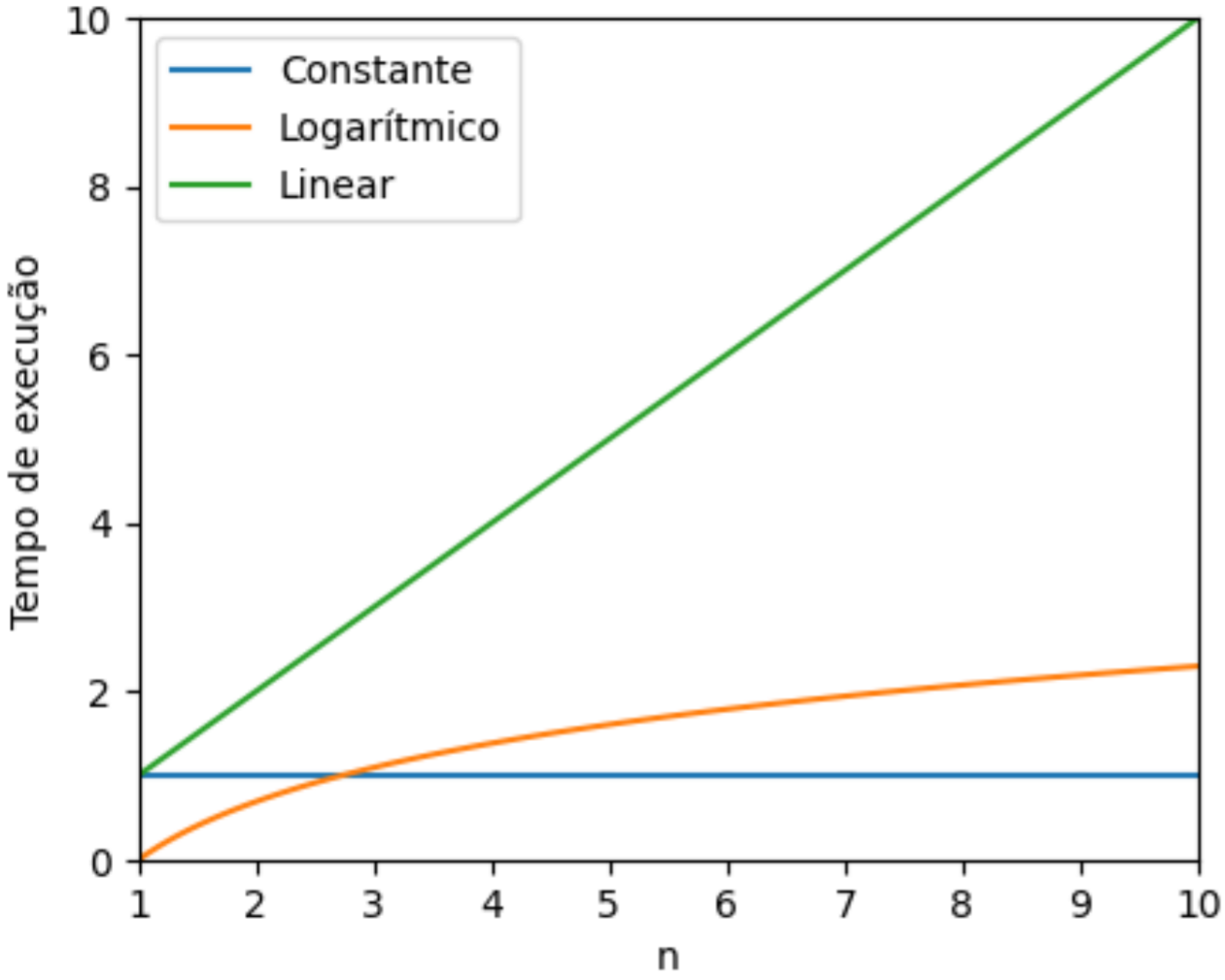
```
import numpy as np # Biblioteca para computacao numerica
import matplotlib.pyplot as plt # Geracao de graficos

n = np.linspace(1, 10, 100) # 100 numeros espacados
labels = ['Constante', 'Logaritmico', 'Linear']
big_o = [np.ones(n.shape), np.log(n), n]  # np.ones(n.
    shape) gera 100 numeros "1".

# Definindo exibicao do grafico
plt.figure(figsize=(5,4)) # Tamanho da figura
plt.ylim(0,10) # Limite do y

for i in range(len(big_o)):
 plt.plot(n, big_o[i], label = labels[i])
plt.legend()
plt.ylabel('Tempo de execucao')
plt.xlabel('n')
plt.show()
```

Gráfico das funções Big-O



Análises

- Função constante é a ideal: o tempo de execução (runtime) independe de n.
- Na logarítmica, o runtime aumenta pouco com o aumento de n.
- Na função linear, o runtime cresce proporcionalmente a n.

Atividade de implementação...

Simule o gráfico das outras funções Big-O citadas, exibindo todas no mesmo gráfico:

- Log-linear: $O(n \log n)$
- Quadrática: $O(n^2)$
- Cúbica: $O(n^3)$
- Exponencial: $O(2^n)$

Gráfico das funções Big-O

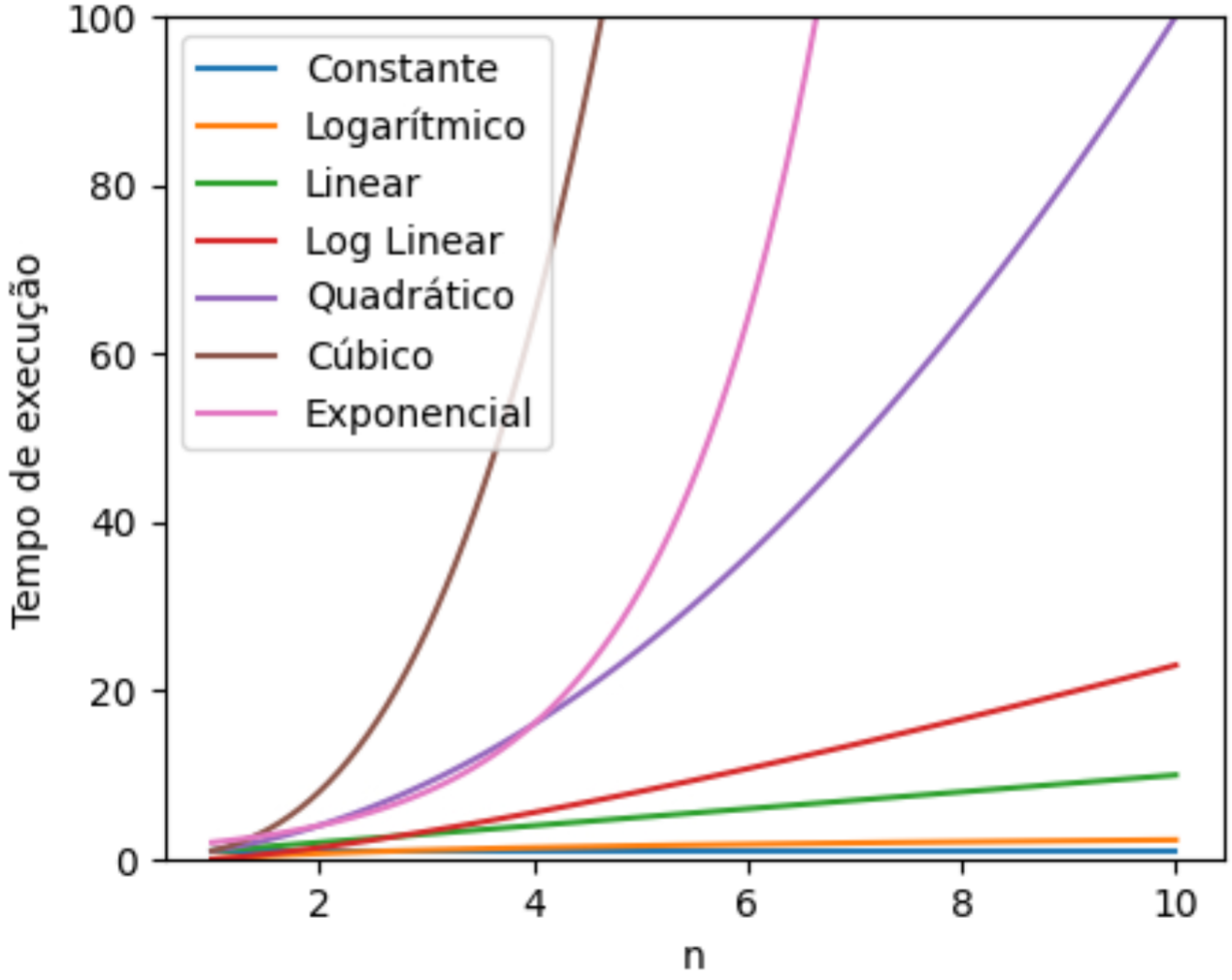
```
import numpy as np # Biblioteca para computacao numerica
import matplotlib.pyplot as plt # Geracao de graficos

n = np.linspace(1, 10, 100) # 100 numeros espacados
labels = ['Constante', 'Logaritmico', 'Linear']
big_o = [np.ones(n.shape), np.log(n), n]  # np.ones(n.
    shape) gera 100 numeros "1".

# Definindo exibicao do grafico
plt.figure(figsize=(5,4)) # Tamanho da figura
plt.ylim(0,10) # Limite do y

for i in range(len(big_o)):
 plt.plot(n, big_o[i], label = labels[i])
plt.legend()
plt.ylabel('Tempo de execucao')
plt.xlabel('n')
plt.show()
```

Gráfico das funções Big-O



Análises

- A log-linear apresenta aumento no runtime em relação às anteriores.
- Os piores casos são as funções quadrática, cúbica (polinomiais) e exponencial.

Exemplo: Big-O constante

```python
# Funcao constante
lista = [1, 2, 3, 4, 5]

# O(1) - Imprimir o primeiro valor de uma lista
def constante(n):
  print(n[0])

# O(2) - Imprimir os dois primeiros valores
def constante2(n):
  print(n[0])
  print(n[1])

constante(lista)
constante2(lista)
```

Exemplo: Big-O linear

```
# Funcao linear
lista = [1, 2, 3, 4, 5]

# O(n) - Imprimir todos os valores de uma lista
def linear(n):
  for i in n:
    print(i)

linear(lista)
```

Exemplo: Big-O polinomial quadrática

```
# Funcao quadratica
# Exemplo: Imprimir todos os n valores de uma lista

def quadratica(n):
  for i in n:
    for j in n:
      print(i, j)

quadratica(lista)
```

Exemplo: Big-O de funções combinadas

```
def combinacao(n):
  print(n[1])

  for i in range(5):
    print(f'teste {i}')

  for i in n:
    print(i)

  for i in n:
    print(i)

  print('teste')
  print('teste')
  print('teste')

combinacao(lista)
```

Exemplo: Big-O de funções combinadas

```python
def combinacao(n):
  print(n[1]) # O(1)

  for i in range(5): # O(5)
    print(f'teste {i}')

  for i in n: # O(n)
    print(i)

  for i in n: # O(n)
    print(i)

  print('teste') # O(1)
  print('teste') # O(1)
  print('teste') # O(1)
```

$$O(1) + O(5) + O(n) + O(n) + O(3) = O(9) + O(2n) = O(n)$$

Questões

- Para que é utilizada a notação Big-O?
- Qual o Big-O dos códigos abaixo?

```
for a in range(n):
  for b in range(n):
    print('teste')
```

```
l = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
for i in range(len(l)):
    print(l[0])
    break
```

```
for i in range(5):
  print(i)
```

Gabarito das questões

- **Função 1** (laços aninhados): $O(n^2)$ — duas estruturas repetitivas sobre n.
- **Função 2** (acesso ao primeiro elemento + `break`): $O(1)$ — executa apenas uma iteração.
- **Função 3** (laço fixo de 5 iterações): $O(1)$ — número constante de operações.